Assignment - python

Q1 - Explain the key features of Python that make it a popular choice for programming

Ans - **Key Features of Python**

Python is one of the most popular programming languages due to its simplicity, flexibility, and powerful features. The main features of Python are as follows:

1. Simple and Easy to Learn

Python has a clear and readable syntax similar to English, which makes it easy for beginners to learn and understand.

2. Interpreted Language

Python is an interpreted language, meaning the code is executed line by line. This makes debugging easier and faster

3. Dynamically Typed

In Python, there is no need to declare the data type of a variable. The type is automatically assigned during runtime, which speeds up coding.

4. Large Standard Library

Python provides a vast standard library with built-in modules and functions, allowing developers to perform many tasks without writing extra code.

5. Platform Independent

Python is a cross-platform language. Programs written in Python can run on different operating systems like Windows, Linux, and macOS without modification.

6. Supports Multiple Programming Paradigms

Python supports different programming styles such as:

- Object-Oriented Programming (OOP)
- Procedural Programming
- Functional Programming

7. Extensive Libraries and Frameworks

Python has a wide range of external libraries and frameworks used for:

- Web development (Django, Flask)
- Data analysis (Pandas, NumPy)
- Machine Learning (TensorFlow)

8. Strong Community Support

Python has a large and active community. This helps in finding solutions, learning resources, and support easily.

9. Automation and Scripting

Python is widely used for automation tasks like file handling, web scraping, and system administration.

10. Versatile and Widely Used

Python is used in many fields such as web development, data science, artificial intelligence, and DevOps.

Q2 - Describe the role of predefined keywords in Python and provide examples of how they are used in a program

Ans - **Role of Predefined Keywords in Python**

Predefined keywords in Python are reserved words that have special meanings and purposes in the language. These keywords are used to define the structure and logic of a Python program. Since they are reserved, they cannot be used as variable names, function names, or identifiers.

Keywords help in writing clear and structured programs by performing specific functions such as defining conditions, loops, functions, classes, and more.

Functions of Keywords with Examples

1. Conditional Statements

Keywords like `if`, `else`, and `elif` are used to make decisions in a program.

```
age = 18
if age >= 18:
   print("Eligible to vote")
else:
   print("Not eligible")
```

2. Looping Statements

Keywords such as `for` and `while` are used to repeat a block of code.

```
for i in range(5):
   print(i)
```

3. Function Definition

The `def` keyword is used to define a function.

Example:

```
def greet():
    print("Hello")
greet()
```

---

4. Boolean Values

Keywords like `True`, `False`, and `None` represent special constant values.

Example:

```
x = True
y = None
```

---

5. Loop Control Statements

Keywords such as `break` and `continue` control the flow of loops.

Example:

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

---

6. Exception Handling

Keywords like `try`, `except`, and `finally` are used to handle errors.

Example:

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Error occurred")
```

Q3 - Compare and contrast mutable and immutable objects in Python with examples

Ans - Mutable vs Immutable Objects in Python

In Python, objects are classified into two types based on whether their values can be changed after creation: **mutable** and **immutable** objects.

1. Mutable Objects

Mutable objects are those whose values can be changed after they are created. This means you can modify, add, or remove elements without creating a new object.

Examples of Mutable Objects:

- List
- Dictionary
- Set

Example:

```
my_list = [1, 2, 3]
my_list[0] = 10
print(my_list)
```

Output:

```
[10, 2, 3]
```

2. Immutable Objects

Immutable objects are those whose values cannot be changed once they are created. If you try to modify them, a new object is created instead.

Examples of Immutable Objects:

- Integer
- Float
- String
- Tuple

```
x = 10
x = x + 5
print(x)
```

Feature	Mutable Objects	Immutable Objects
Modification	Can be changed	Cannot be changed
Memory Usage	Same object is modified	New object is created
Examples	List, Set, Dictionary	Int, Float, String, Tuple
Performance	Faster for changes	Safer and more secure

Q4 - Discuss the different types of operators in Python and provide examples of how they are used

Ans - **Types of Operators in Python**

Operators in Python are special symbols used to perform operations on variables and values. Python provides several types of operators, which are explained below with examples.

1. Arithmetic Operators

Operator	Meaning	Example
`+`	Addition	`a + b`
`-`	Subtraction	`a - b`
`*`	Multiplication	`a * b`
`/`	Division	`a / b`
`%`	Modulus	`a % b`
`**`	Exponentiation	`a ** b`
`//`	Floor Division	`a // b`

```
a = 10
b = 3
print(a + b)   # 13
print(a // b)  # 3
```

2. Comparison (Relational) Operators

These operators compare two values and return `True` or `False`.

Operator	Meaning
`==`	Equal to
`!=`	Not equal to
`>`	Greater than
`<`	Less than
`>=`	Greater or equal
`<=`	Less or equal

```
x = 5
y = 10
print(x < y)   # True
print(x == y)  # False
```

3. Logical Operators

These are used to combine conditional statements.

Operator	Meaning
`and`	True if both are true
`or`	True if at least one is true
`not`	Reverses the result

```
a = True
b = False
print(a and b)  # False
print(a or b)   # True
```

4. Assignment Operators

These operators are used to assign values to variables.

Operator	Example	Meaning
`=`	`x = 5`	Assign value
`+=`	`x += 2`	Add and assign
`-=`	`x -= 2`	Subtract and assign
`*=`	`x *= 2`	Multiply and assign

```
x = 5
x += 3
print(x)  # 8
```

5. Bitwise Operators

These operators perform operations on binary numbers.

Operator	Meaning
`&`	AND
`` ` ``	`` ` ``
`^`	XOR
`~`	NOT
`<<`	Left Shift
`>>`	Right Shift

```
a = 5   # 101
b = 3   # 011
print(a & b)  # 1
```

6. Membership Operators

These are used to test if a value exists in a sequence.

Operator	Meaning
`in`	Present in sequence
`not in`	Not present

```
my_list = [1, 2, 3]
print(2 in my_list)  # True
```

7. Identity Operators

These operators compare the memory location of two objects.

Operator	Meaning
`is`	Same object
`is not`	Different object

```
a = [1, 2]
b = a
print(a is b)  # True
```

Q5 - Explain the concept of type casting in Python with examples

Ans - Type Casting in Python

Type casting in Python refers to the process of converting one data type into another. It is useful when we need to perform operations that require compatible data types.

Python provides two types of type casting:

1. **Implicit Type Casting**
2. **Explicit Type Casting**

---

1. Implicit Type Casting

Implicit type casting is automatically performed by Python without the programmer's intervention. It usually occurs when Python converts a smaller data type into a larger data type to avoid data loss.

a = 5 # integer

b = 2.5 # float

c = a + b # Python automatically converts 'a' to float

print(c)

print(type(c))

2. Explicit Type Casting

Explicit type casting is done manually by the programmer using built-in functions like `int()`, `float()`, `str()`, etc.

Common Type Casting Functions:

- `int()` → converts to integer
- `float()` → converts to float
- `str()` → converts to string

(a) Converting float to int

```
x = 5.8
y = int(x)
print(y)
```

Output:

```
5
```

(b) Converting int to float

```
a = 10
b = float(a)
print(b)
```

Output:

```
10.0
```

(c) Converting int to string

```
num = 100
text = str(num)
print(text)
print(type(text))
```

Important Points

- Implicit casting is automatic and safe.
- Explicit casting is done by the programmer when needed.

- Improper casting (like converting letters to int) can cause errors.

Q6- How do conditional statements work in Python? Illustrate with examples

Ans - Conditional Statements in Python

Conditional statements in Python are used to make decisions in a program. They allow the program to execute certain blocks of code based on whether a condition is **True** or **False**.

Python uses keywords like `if`, `elif`, and `else` to implement conditional statements.

Working of Conditional Statements

- The `if` statement checks a condition.
- If the condition is **True**, the block of code inside `if` is executed.
- If the condition is **False**, the program checks the `elif` (else if) condition (if present).
- If none of the conditions are true, the `else` block is executed.

Types of Conditional Statements

1. Simple `if` Statement

Executes a block of code only if the condition is true.

Example:

```
x = 10
if x > 5:
    print("x is greater than 5")
```

2. `if-else` Statement

Executes one block if the condition is true, otherwise executes another block.

Example:

```
age = 16
if age >= 18:
    print("Eligible to vote")
else:
    print("Not eligible to vote")
```

3. `if-elif-else` Statement

Used when multiple conditions need to be checked.

```
marks = 75
```

```
if marks >= 90:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

4. Nested `if` Statement

An `if` statement inside another `if` statement.

```
num = 10


if num > 0:
    if num % 2 == 0:
        print("Positive even number")
```

Q7- Describe the different types of loops in Python and their use cases with examples

Ans - **Loops in Python**

Loops in Python are used to execute a block of code repeatedly as long as a condition is satisfied. They help reduce code repetition and make programs more efficient.

Python mainly provides two types of loops:

1. **`for` loop**
2. **`while` loop**

---

1. `for` Loop

The `for` loop is used to iterate over a sequence such as a list, tuple, string, or range.

Use Cases:

- When the number of iterations is known
- Iterating through elements of a collection

Example:

```
for i in range(5):
    print(i)
```

Output:

```
0
```

```
1
2
3
4
```

---

2. `while` Loop

The `while` loop is used to execute a block of code as long as a condition remains true.

Use Cases:

- When the number of iterations is not known
- When the loop depends on a condition

Example:

```
i = 0
while i < 5:
    print(i)
    i += 1
```

Output:

```
0
1
2
3
4
```

Loop Control Statements

Python also provides statements to control the flow of loops:

1. `break`

Used to exit the loop immediately.

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

---

2. `continue`

Skips the current iteration and moves to the next.

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

---

3. `pass`

Acts as a placeholder and does nothing.

```
for i in range(3):
    pass
```

Key Differences Between `for` and `while` Loop

Feature	`for` Loop	`while` Loop
Iterations	Known in advance	Not known in advance
Condition	Based on sequence	Based on condition
Usage	Iterating collections	Condition-based execution